

Timers e Interrupciones en STM32

UCC – Software de Sistemas Embebidos - 2026

Dr. Ing. Agustín Laprovitta (alaprovitta@ucc.edu.ar)
Ing. Martín Molina (2369147@ucc.edu.ar)

TRABAJO PRÁCTICO N°2

Objetivos de Aprendizaje

Al finalizar este trabajo práctico, el alumno será capaz de:

1. Comprender la arquitectura del sistema de reloj (RCC) y el cálculo de la frecuencia de conteo mediante el Prescaler (PSC) y el Auto-Reload Register (ARR).
2. Configurar un Timer en modo Time Base (Base de Tiempo) y gestionar eventos de desbordamiento (Update Event) mediante Interrupciones por Hardware (NVIC).
3. Generar señales de Modulación por Ancho de Pulso (PWM) para el control de intensidad luminosa en LEDs y tono en Buzzer pasivo.
4. Implementar la lectura de eventos externos mediante Input Capture (Captura de Entrada) o temporización precisa por hardware.
5. Comprender la diferencia entre procesamiento por encuesta (polling) e interrupciones (IRQ), escribiendo código limpio dentro del callback de HAL (HAL_TIM_PeriodElapsedCallback).

Conceptos Teóricos Clave

Antes de iniciar la práctica, se repasan las fórmulas fundamentales para la temporización del Timer:

$$f_TIMER_CNT = f_TIM_CLK / (PSC + 1)$$

$$T_INTERRUPCIÓN = [(PSC + 1) \times (ARR + 1)] / f_TIM_CLK$$

- **f_TIM_CLK:** Frecuencia del bus APB que alimenta al Timer (ej. 72 MHz en F103, 84 MHz / 168 MHz en F407).
- **PSC (Prescaler):** Divisor de frecuencia del reloj principal del Timer.
- **ARR (Auto-Reload Register):** Valor máximo de conteo del Timer antes de reiniciar a 0 y generar la interrupción (Update Event).

Consignas del Trabajo Práctico

Parte 1: Base de Tiempo e Interrupción por Desbordamiento

Objetivo: Liberar al CPU del bucle while(1) utilizando interrupciones periódicas por hardware.

1. **Configuración STM32CubeMX:** Configure TIM2 en modo Internal Clock. Ajuste PSC y ARR para lograr una interrupción por desbordamiento exactamente cada 500 ms. En la pestaña NVIC Settings, habilite la interrupción TIM2 global interrupt.
2. **Firmware & Callback:** En main(), inicie el Timer en modo interrupción con HAL_TIM_Base_Start_IT(&htim2);. Implemente la función callback oficial de la HAL HAL_TIM_PeriodElapsedCallback() para conmutar el estado de un LED (por ejemplo PA5 en F103 o PD12 en F407).
3. **Verificación:** Compruebe que el bucle principal while(1) permanece completamente vacío (o ejecutando tareas secundarias) mientras el LED parpadea a 1 Hz de forma precisa.

Parte 2: Modulación por Ancho de Pulso (PWM) y Control Dimmer

Objetivo: Generar señales analógicas equivalentes mediante PWM para control de intensidad y tono.

1. **Configuración STM32CubeMX:** Configure TIM3_CH1 en modo PWM Generation CH1. Calcule PSC y ARR para obtener una frecuencia fija de PWM de 1 kHz.
2. **Firmware & Control:** Inicie el canal PWM con HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);. Mediante una base de tiempo no bloqueante, varíe dinámicamente el registro Compare (CCR1) entre 0 y ARR para generar un efecto de respiración (Breathing LED) o dimmer suave.
3. **Extensión Opcional:** Envíe la señal PWM al pin del Buzzer pasivo para generar tonos audibles modificando la frecuencia o el ciclo de trabajo (Duty Cycle).

Parte 3: Medición de Tiempo mediante Input Capture

Objetivo: Medir la duración de eventos externos o pulsos sin carga sobre la CPU.

1. **Configuración STM32CubeMX:** Configure TIM4_CH1 en modo Input Capture direct mode. Habilite las interrupciones en el NVIC para TIM4 global interrupt.
2. **Firmware & Medición:** Configure la detección de flancos (ascendente/descendente) en el pin del pulsador S1. Implemente el callback HAL_TIM_IC_CaptureCallback() para registrar el tiempo que un usuario mantiene presionado el pulsador en milisegundos.
3. **Salida:** Muestre o verifique el valor de tiempo medido utilizando la ventana Live Expressions durante el debugging.

Parte 4: Desafío de Integración (Sistema Multitarea por Timers)

Objetivo: Coordinar múltiples temporizadores e interrupciones en una arquitectura multitarea por hardware.

Escriba un programa que no utilice HAL_Delay() ni HAL_GetTick() en el bucle main(), delegando toda la gestión temporal a los Timers de hardware:

- **TIM2 (Base de Tiempo 500 ms):** Incrementa un contador global y refresca el valor en el Display de 7 segmentos / consola.
- **TIM3 (PWM 1 kHz):** Controla el brillo del LED 1. El ciclo de trabajo (Duty Cycle) se incrementa un 10% con cada pulsación de S1.
- **TIM4 (Base de Tiempo 5 s):** Genera un beep en el buzzer activando durante 100 ms la salida, controlado exclusivamente por la interrupción del Timer.

Preguntas de Cierre para profundización de aprendizajes

- ¿Por qué debemos sumar 1 a los valores de PSC y ARR al aplicar las fórmulas de cálculo de frecuencia?
- ¿Qué consecuencias técnicas tiene escribir código bloqueante (ej. HAL_Delay o printf) dentro de HAL_TIM_PeriodElapsedCallback?
- Si la frecuencia de reloj del sistema cambia de 72 MHz a 168 MHz en STM32CubeMX, ¿qué registros del Timer deben modificarse para conservar la misma base de tiempo?
- ¿Cuál es la diferencia conceptual y funcional entre el registro ARR (Auto-Reload Register) y el registro CCR (Capture/Compare Register)?

Consigna para la Entrega de Informe (Aula Virtual)

Formato y Condiciones de Entrega

- Formato de archivo: PDF (TP_GPIO_Apellido_Nombre.pdf).
- Extensión máxima: 3 páginas (formato Ficha Técnica o *Reporte Ejecutivo*).
- Plazo de entrega: 04/09/2026.
- Modalidad: Individual.

Estructura y Contenido del Informe

1. Encabezado

- Nombre completo del estudiante.
- Asignatura y fecha de realización.
- Modelo de microcontrolador utilizado (STM32F103RB o STM32F407).

2. Cálculos Justificados de Temporización (Timers)

Presente el procedimiento matemático utilizado para calcular los registros de los Timers solicitados:

Timer	Frecuencia de Bus (f_{TIM_CLK})	Tiempo/frecuencia deseada	Prescaler PSC	Auto-Reload ARR
TIM2 (Base de tiempo 500 ms)		2 Hz		
TIM3 (Frecuencia PWM)		1kHz		

Nota: Muestre la fórmula aplicada especificando por qué se resta/suma el valor 1 en los registros PSC y ARR.

3. Configuración de Interrupciones (NVIC) y PWM

- Tabla de Interrupciones: Indique las interrupciones habilitadas en CubeMX, su prioridad asignada en el NVIC y la función Callback correspondiente.
- Configuración PWM: Indique el canal de Timer configurado para PWM (ej. TIM3_CH1) y el rango de valores asignados al registro CCR para controlar el brillo o tono.

Evidencia de Funcionamiento

- Explicación de las tareas realizadas y los desafíos encontrados.
- Adjunte un Video corto (link de Drive/YouTube) que muestre el sistema operando de cada ejercicio. Revisión del docente en clases.

Análisis Técnico y Conclusión Breve (Responda en 3 o 4 líneas)

- ¿Qué habría ocurrido con la precisión del sistema o la respuesta a otros eventos si la temporización de 500 ms y la variación del PWM se hubieran programado utilizando HAL_Delay() dentro de un bucle while(1) en lugar de interrupciones de Timer?

Rúbrica de Evaluación

Criterio	Sobresaliente (10-9)	Satisfactorio (8-7)	En Proceso (6-4)	Incompleto (3-1)
1. Cálculos de PSC y ARR	Aplica las fórmulas con precisión exacta para F103 y F407.	Cálculos correctos con pequeños errores de +1 por registro.	Cálculos erróneos; la temporización difiere del objetivo.	No sabe calcular PSC ni ARR.
2. Uso de Interrupciones	Habilita NVIC e implementa callbacks de manera limpia.	Implementa interrupción con código algo extenso en callback.	El código se congela o no limpia flags adecuadamente.	No logra configurar interrupciones por hardware.
3. Generación de PWM	Configura PWM y varía el CCR suavemente sin parpadeos.	Genera PWM pero no logra variación dinámica desde código.	Frecuencia incorrecta o genera parpadeos visibles.	No logra generar la señal PWM.
4. Integración y Multitarea	Bucle main libre de bloqueos; temporización 100% por hardware.	Integra Timers pero depende parcialmente de polling.	Timers interfieren entre sí por mala prioridad de NVIC.	No integra más de un Timer.

Sistemas Embebidos — Microcontroladores STM32F103 & STM32F407

Placas Compatibles: STM32 Nucleo-F103RB / STM32F407 (Discovery / Black Board)

Entorno de Desarrollo: STM32CubeIDE (C con librería STM32 HAL)

Componentes: Cables / LEDs / Pulsador / Buzzer / Shield o protoboard

Tiempo Estimado: 1 o 2 sesiones